

SIH26181 — Master Theory Notes & Judge Defense Companion

AI-Powered Personal Health Companion & Edge Disaster Monitor

FPGA-Accelerated, Cloud-Free Biometric & Disaster Resilience Engine

1. THE ULTIMATE PITCH GUIDE (Executive Summary)

If you have 3 minutes to pitch this to a judge, hit these exact points in this order:

1. What are we solving? (The Problem)

In 2024 alone, extreme heatwaves in India claimed over 700 lives in just three months, while 380 million outdoor workers (75% of the workforce) faced severe risk of heatstroke and lost wages. The problem is that current early-warning systems only monitor the *environment* (e.g., weather apps), leaving these vulnerable workers guessing how much danger their own bodies are actually in.

2. How does it relate to the SIH Problem Statement? (Qualcomm Hardware Challenge)

Smart India Hackathon Problem **SIH26181** asks for innovative hardware solutions leveraging Qualcomm platforms for societal impact. We built a hardware-accelerated, edge-AI wearable that acts as a personal disaster monitor. It proves custom hardware offloading (FPGA to Snapdragon transition) while solving a massive societal problem: protecting vulnerable populations during climate crises completely offline.

(*HARDWARE WOW-FACTOR*): Most hackathon health wearables run everything in software on a standard microcontroller. We went further—we designed custom digital hardware circuits (in Verilog) that process heartbeat signals physically at the silicon level, not in code.

3. Who is doing this in the world? (The Competitors)

Currently, the market is fractured:

- **Fitness Wearables (Apple Watch / Garmin):** They track your heart rate but have absolutely no idea if you are standing in heavy wildfire smoke or 45°C heat. They lack environmental context.
- **Environmental Sensors (PurpleAir / Weather apps):** They tell you the air quality is bad, but they don't know if *your* specific lungs are failing because of it. Plus, they rely on cloud/internet connections which fail during disasters.

4. How are we improving / What are we fixing? (Our Novelty)

We are fixing the "cloud dependency" and the "context gap."

- **The Fix:** We built a **Hardware-Accelerated Sensor Fusion Engine**. We don't just put an AQI sensor and a Heart Rate sensor in the same box; our TinyML Neural Network fuses them together locally on the device.
- **The Improvement:** If the air is toxic ($PM_{2.5} > 300$) and your blood oxygen (SpO_2) suddenly drops while your heart rate spikes, our device doesn't just show you raw numbers—it instantly calculates your exact physiological strain (PRSI) and sounds a critical alarm, completely offline.

[🖱️ TRANSITION TO DEMO]

(At this exact moment, point to your hardware. If you only have a digital simulation right now, open it up).

"Let me show you exactly what happens in our simulation when the PM2.5 and Heart Rate spike simultaneously..."

5. Who is this for? (Target Audience)

- **Vulnerable Demographics:** The elderly, asthma patients, and individuals with cardiovascular issues who are highly susceptible to sudden heatwaves or pollution spikes.
- **Frontline & Outdoor Workers:** Firefighters, construction workers, and disaster-relief teams who need offline, real-time alerts before they collapse from heatstroke or toxic smoke inhalation.

6. The Vision (Closing the Pitch)

We've proven this works today on custom FPGA logic. By migrating to Qualcomm's Snapdragon platform, this transforms from a prototype into a mass-producible, ultra-low-power wearable capable of protecting millions during the next climate disaster.

2. SYSTEM ARCHITECTURE — EXPLAINED SIMPLY

The Zynq-7000 (FPGA + ARM CPU) Setup

A **SoC** (System on Chip) like the Zynq-7000 is special because it combines two different types of brains on a single chip:

- **The CPU (ARM Cortex-A9):** Think of this as the "manager." It's great at running software, making complex decisions (like our AI model), and coordinating the system, but it processes things one step at a time.
- **The FPGA Fabric:** Think of this as a "custom factory assembly line." Instead of running software, you physically wire the hardware to do exactly one job incredibly fast.

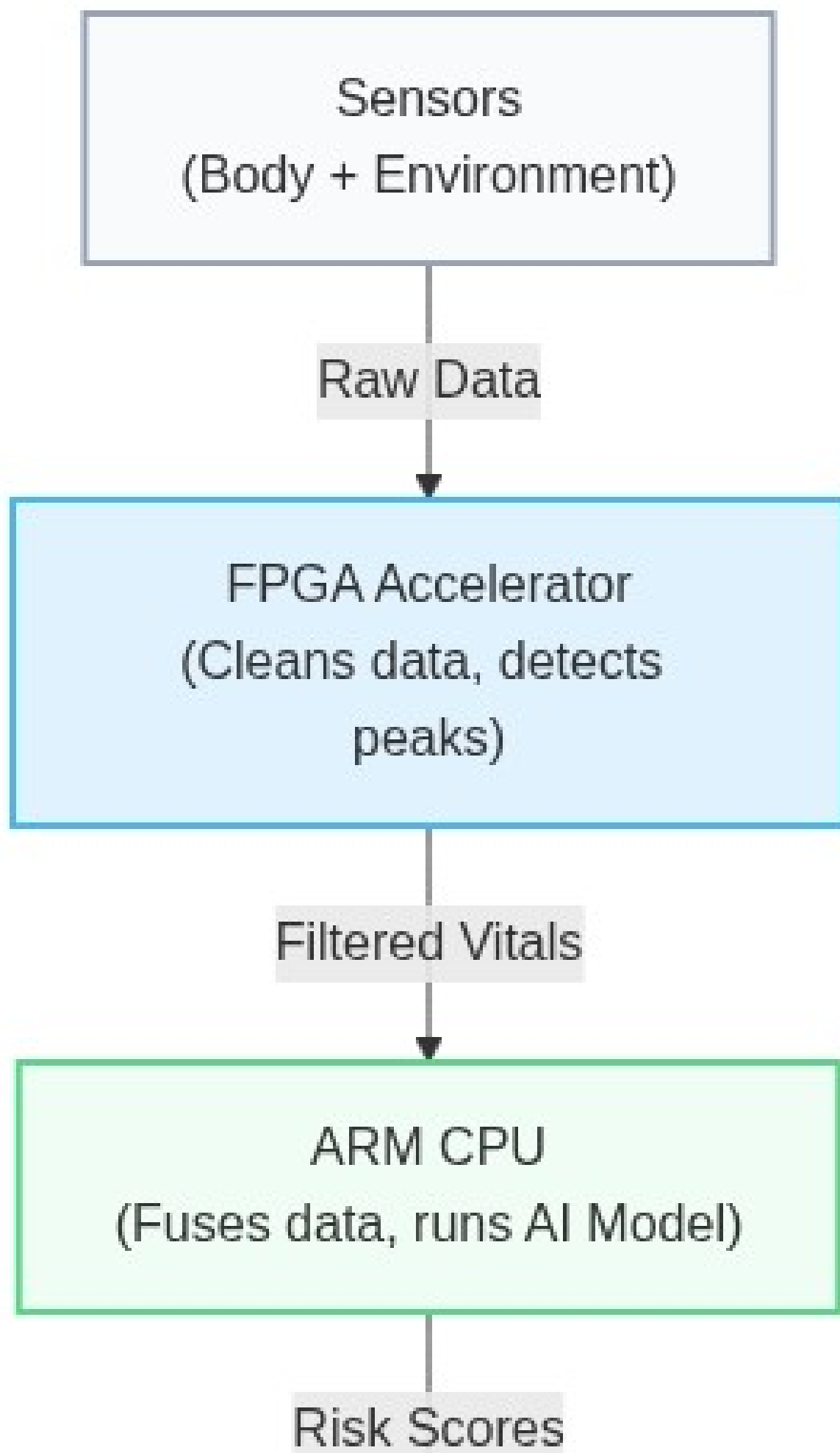
Why this combination matters:

Processing raw heartbeat (PPG) data requires constant, repetitive math (filtering noise, finding peaks) thousands of times a second. If the CPU did this, it would be overwhelmed and drain the battery. By building a custom "factory" on the FPGA to handle the repetitive heartbeat math, we free up the CPU to run the complex Neural Network and drive the display.

Migration Path to Qualcomm Snapdragon Wear (QCS6490)

- **What changes:** We move from an FPGA-based architecture to Qualcomm's highly integrated mobile platform, leveraging their dedicated DSP (Hexagon) or NPU (Neural Processing Unit) instead of our custom FPGA logic.
- **What stays the same:** The sensor suite, the C-based Neural Network inference engine, and the overall data flow (Sensors → Accelerator → CPU → Display).
- **Why migrate:** The Zynq is great for prototyping custom hardware, but it's too large and power-hungry for a wrist-worn wearable. The Snapdragon Wear platform is purpose-built for ultra-low-power wearables, offering commercial-grade battery life and a smaller physical footprint.

Data-Flow Diagram (Simple)





OLED Display
(Shows warnings offline)

3. EACH SENSOR — WHAT IT DOES AND WHY WE CHOSE IT

- **MAX30102 (PPG/SpO2):**

- *What it is:* A pulse oximeter and heart-rate sensor.
- *How it works:* It shines a red and infrared LED into your skin and measures how much light bounces back to a photodiode. Since oxygen-rich blood absorbs light differently than oxygen-poor blood, we can calculate **SpO2** (blood oxygen). The physical pulsing of blood with each heartbeat changes the light reflection, giving us the **PPG** (Photoplethysmography) waveform to find heart rate.

- **BME280 (Temperature/Humidity):**

- *Note:* MAX30102 includes an on-chip die temperature sensor which serves as a proxy for skin microclimate temperature during cold exposure assessment.
- *Why it matters:* To predict heatstroke, you need to know the ambient temperature and humidity. High humidity prevents sweat from evaporating, drastically increasing heat exhaustion risk even at lower temperatures.

- **PMS5003 (PM2.5):**

- *What it is:* A particle sensor that detects microscopic dust and smoke (PM2.5 means particles smaller than 2.5 micrometers).
- *Why it matters:* PM2.5 particles are small enough to cross into the bloodstream. During wildfires or severe smog, this sensor combined with falling blood oxygen (SpO2) tells us the user is in severe respiratory distress.

- **SSD1306 OLED:**

- *Why it's included:* It provides immediate, on-device visual feedback (heart rate, risk level alerts) to the user without needing a smartphone app.

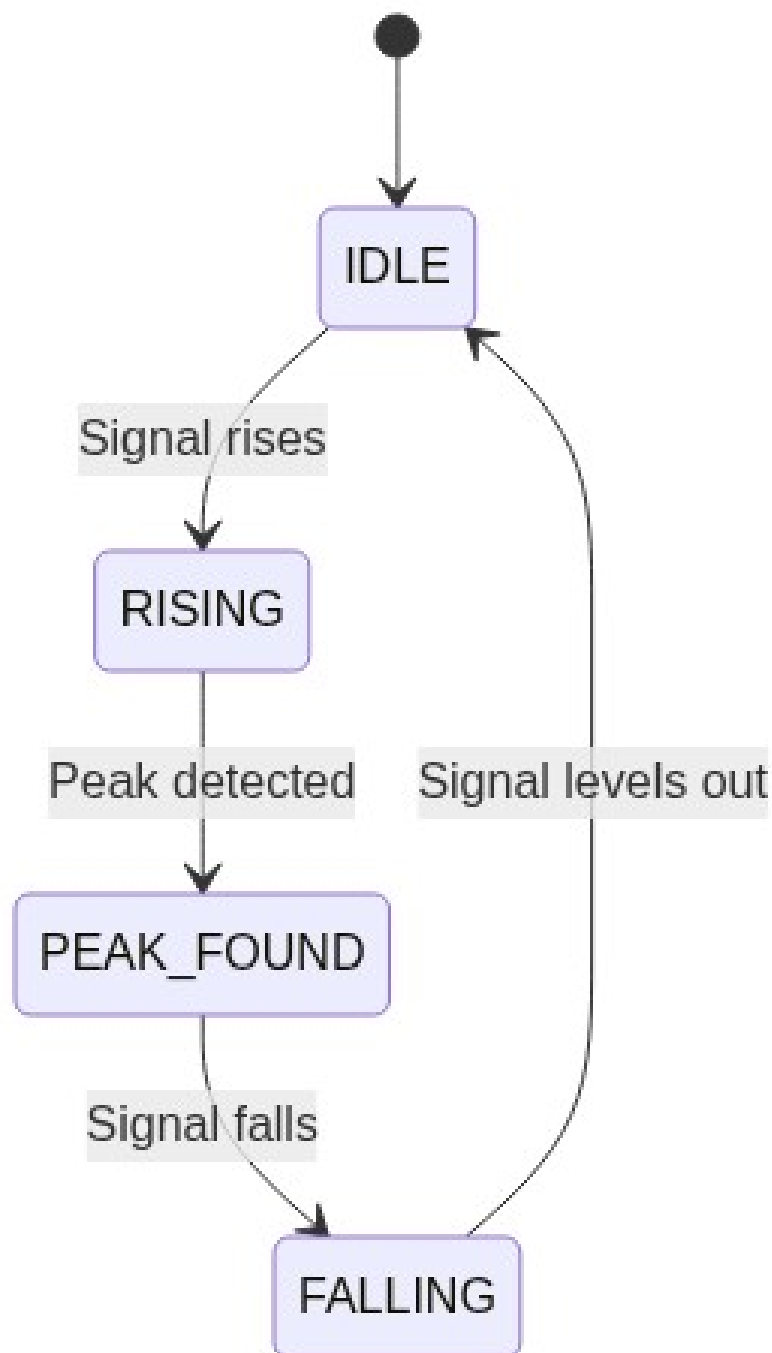
4. THE CUSTOM VERILOG / RTL — EXPLAINED FOR A NON-HARDWARE PERSON

We wrote custom hardware code (**RTL** or **Verilog**) to create our own chip logic inside the FPGA.

- **axippgaccelerator.v:**

- *What it is:* An **AXI accelerator**. AXI is just the standard "bridge" that lets our custom FPGA hardware talk to the ARM CPU.

- *Why we need it:* Instead of the CPU reading raw sensor data and struggling to clean it, the sensor data goes straight to our custom hardware block. The CPU only receives the final, cleaned results.
- **movingaverage8tap.v:**
 - *What it is:* A digital filter that smooths out the raw heartbeat signal. When you move your wrist, the sensor gets noisy.
 - *Why 8-tap and O(1):* "8-tap" means it averages the last 8 readings. "O(1)" means it does this math instantly in one clock cycle by keeping a running total, rather than recalculating everything from scratch. This is hyper-efficient.
- **ppgpeakdetector.v:**
 - *What it is:* A 4-state **FSM** (Finite State Machine). Think of an FSM as a flow chart built into hardware.
 - *The States:*



- *Why it matters:* By counting the exact time between each `PEAK_FOUND` state, we determine the user's heart rate and HRV (Heart Rate Variability) instantly in hardware.

5. THE SOFTWARE / ML SIDE — EXPLAINED SIMPLY

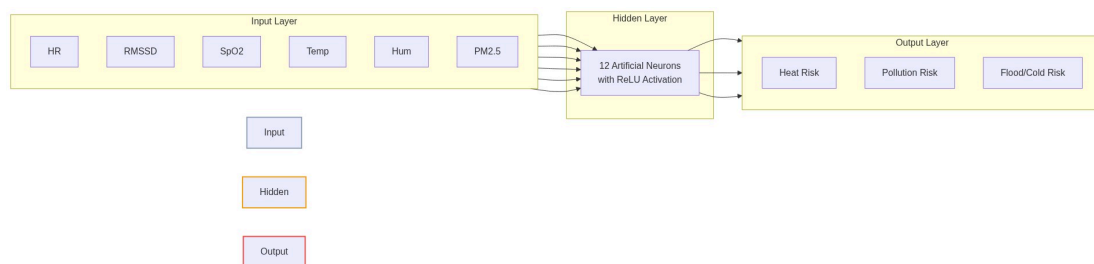
Our latest software update (`nnriskmodel.c` and `disasterriskengine.c`) introduces a **Hybrid Risk Assessment Engine**.

How we calculate risk:

We take 6 vital signs and environmental readings: Heart Rate, RMSSD (a measure of stress/HRV), SpO2, Ambient Temp, Humidity, and PM2.5.

We process these in two ways:

- 1. Rule-Based Engine (`disasterriskengine.c`):** This calculates medical indices like the Cardio-Thermal Strain Index (CTSI) for heatwaves and the Pollution Respiratory Strain Index (PRSI) for air quality. Just like CTSI combines Temp + HR for heat strain, PRSI combines PM2.5 + SpO2 for pollution strain. For example, if temperature > 40°C AND heart rate > 120 bpm, CTSI immediately flags a Critical Heat Risk.
- 2. The Neural Network (`nnriskmodel.c`):** We implemented a lightweight, custom Artificial Intelligence model (a Multi-Layer Perceptron) entirely in C.



- *How it works:* The 6 inputs are normalized (scaled between 0 and 1) and passed through a "hidden layer" of 12 artificial neurons. It uses mathematical functions (ReLU and Sigmoid) to weigh how different factors interact. For instance, high PM2.5 alone is bad, but high PM2.5 *combined* with a dropping SpO2 and high heart stress triggers a much higher risk score.
- *The Output:* It generates three risk scores from 0.0 to 1.0 for Heat, Pollution, and Flood/Cold exposure. A score above 0.70 triggers a CRITICAL alert.
- **Training via "Knowledge Distillation":** If a judge asks how this TinyML model was trained, use this "Teacher vs. Student" analogy:
 - *The Teacher (Rule-Based Engine):* Our hard-coded medical equations act as the Teacher. It is perfectly safe and accurate, but running all those nested `if/else` checks drains the battery.
 - *The Student (Neural Network):* The AI starts completely blank. It uses far less battery and runs much faster than the Teacher.
 - *The Homework:* We generated 50,000 synthetic flashcards of extreme scenarios (e.g., HR=140, Temp=47°C, PM2.5=400).
 - *The Grading:* The Teacher graded the flashcards with the "correct" risk scores. The Student tried to guess the answers. Every time the Student guessed wrong, we adjusted its neurons (via Gradient Descent) so it learned the Teacher's medical rules.

- *The Final Exam:* We tested the Student on 5,000 brand-new scenarios. It guessed the exact same critical risk category (Normal, Moderate, High, Critical) as the heavy Rule-Based Engine with **83.44% classification accuracy**. We effectively compressed a heavy medical rule-book into a tiny, ultra-fast neural network.
-

6. WHY THESE DESIGN CHOICES

- **Why FPGA acceleration instead of doing everything in software?**

Battery life and responsiveness. Doing signal processing on a CPU wastes power. Custom hardware does the exact same math in a fraction of a microsecond using practically zero power.

- **Why a hybrid rule-based + ML approach instead of pure ML?**

Safety and explainability. Pure AI can sometimes be a "black box" and make weird mistakes. If the AI fails, our hard-coded rule-based engine acts as a safety net ensuring obvious critical conditions (like SpO2 falling below 85%) trigger an alarm immediately.

- **Why edge/on-device processing instead of sending data to the cloud?**

During natural disasters (floods, hurricanes), cellular towers and internet go down. A disaster monitor that relies on the cloud is useless when you need it most. Our device saves lives completely offline.

- **Why these specific sensors?**

They are the exact minimum required to triangulate the three deadliest climate threats: Heat (Ambient Temp/Humidity + HR), Wildfire/Pollution (PM2.5 + SpO2), and Cold/Floods (Skin temp proxy + HRV stress).

- **Why Zynq now vs Qualcomm Snapdragon later?**

The Zynq FPGA allowed us to quickly design and prove our custom hardware algorithms (like the O(1) filter and peak detector). Migrating to Qualcomm allows us to turn this proven logic into a mass-market, consumer-friendly smartwatch form factor.

7. LIKELY JUDGE QUESTIONS — WITH SUGGESTED ANSWERS

1. Technical: "Why use an FPGA for this? Wouldn't a cheap microcontroller work?"

Suggested Answer: "A standard microcontroller *could* do it, but at the cost of battery life. By using an FPGA (or dedicated DSP on Snapdragon), we offload the continuous, heavy math of filtering heartbeat signals into hardware. This allows the main CPU to sleep longer, drastically extending battery life, which is critical for a wearable."

2. Technical: "How does your peak detector handle motion artifacts (user moving their arm)?"

Suggested Answer: "Our hardware moving-average filter smooths out high-frequency noise, but for severe motion, we rely on our Neural Network's HRV (Heart Rate Variability) input. If motion heavily corrupts the signal, the HRV becomes erratic, and our hybrid engine drops the confidence score rather than triggering a false alarm."

3. Feasibility: "How does this scale to the Snapdragon platform?"

Suggested Answer: "The architecture is highly portable. The hardware blocks we built in Verilog for the FPGA map perfectly to the Hexagon DSP on the Snapdragon platform. Our C-based Neural Network is completely hardware-agnostic and will run natively on Qualcomm's NPU for even better power efficiency."

4. Real-world: "How is this different from an Apple Watch or Fitbit?"

Suggested Answer: "Consumer smartwatches track fitness; they don't know if you're standing in 45°C heat or heavy wildfire smoke. We fuse environmental data *with* biometric data locally. An Apple Watch tells you your heart rate is high; our device tells you your heart rate is high *because* the humidity is preventing sweat evaporation, and you are 10 minutes away from heatstroke."

5. Weaknesses: "What happens if a sensor fails? Do you get false positives?"

Suggested Answer: "That's exactly why we use a hybrid model. If the PM2.5 sensor breaks and reads maximum pollution, our system checks your blood oxygen and heart rate. If your vitals are perfectly normal, the Neural Network realizes it's likely a sensor error and suppresses a critical panic alarm."

6. Weaknesses: "Your NN was trained on synthetic data. How do you know it generalizes to real patients?"

Suggested Answer: "We used synthetic data to bootstrap training given ethical/access constraints on real patient vitals during hackathon timelines, achieving ~83.4% classification accuracy on that set. However, the rule-based engine acts as our safety net specifically because we don't yet trust the NN's real-world generalization. Before making real-world claims, our explicit next step is to validate the model against IRB-approved wearable datasets like PhysioNet or WESAD."

7. Novelty: "What's stopping someone from just gluing a Fitbit and an AQI sensor together?"

Suggested Answer: "A Fitbit and an AQI sensor can't talk to each other in real-time, especially without a cloud connection. Our novelty is the hardware-accelerated real-time fusion of this data locally at the edge. We don't just show two numbers; we compute the exact physiological strain caused by the environment, instantly, completely offline."

8. GLOSSARY

- **FPGA (Field Programmable Gate Array):** A blank-slate chip that you can program to become custom digital hardware circuitry.
 - **RTL (Register-Transfer Level):** The design abstraction used to write hardware code; effectively synonymous with the Verilog code we wrote.
 - **AXI:** A standardized communication bridge (bus) that allows the ARM CPU to talk to our custom FPGA hardware.
 - **FSM (Finite State Machine):** A hardware design concept that acts like a flowchart, moving from one state to another (e.g., from IDLE to RISING) based on inputs.
 - **PPG (Photoplethysmography):** The optical measurement of blood volume changes in your blood vessels, used to detect heartbeats.
 - **SpO2:** Peripheral capillary oxygen saturation; a measure of how much oxygen is in your blood.
 - **TinyML:** A field of machine learning focused on running AI models on highly constrained, low-power embedded devices instead of large servers.
 - **SoC (System on Chip):** A single integrated circuit that contains an entire computer system, like the Zynq which holds both an ARM CPU and an FPGA.
-

9. PITCH ANALOGIES & HARDWARE EXPLAINER CHEAT-SHEET

If a judge asks you to explain the deeper hardware engineering concepts, use these simple analogies and plain-English file breakdowns.

Killer Analogies

1. The "CEO and the Assembly Line" Analogy

- **What it explains:** Why you used an FPGA (Hardware Acceleration) instead of just using a standard microcontroller to do everything in software.
- **How to pitch it:** *"Think of the CPU as a brilliant CEO sitting at a desk. If you ask the CEO to filter thousands of raw heartbeat signals a second, they can do it, but they have to look at them one by one. The CEO will get exhausted, drain our battery, and have no time to make high-level decisions. Instead, we used the FPGA fabric to build an **automated assembly line**. We physically wired a digital conveyor belt that filters the heartbeat noise instantly as it passes through, using almost zero power. The assembly line just hands the CEO a final report that says: 'The heart rate is 120.' Now, the CEO has all their energy freed up to run our Neural Network."*

2. The "Elevator Weight" Analogy

- **What it explains:** How your custom hardware moving-average filter runs in **O(1)** time complexity (which is extremely impressive to software and hardware engineers).
- **How to pitch it:** *"When a user moves their wrist, the heart-rate sensor gets noisy, so we have to continuously average the last 8 readings. Imagine finding the average weight of 8 people in an elevator. Every time a new person steps in and the oldest person steps out, a standard software algorithm makes all 8 people step on the scale again, adds them up, and divides. That wastes CPU cycles. Our custom O(1) hardware filter is like a smart scale. It just remembers the old total, subtracts the weight of the person leaving, and adds the weight of the person entering. It does this instantly, in a single clock cycle. It takes the exact same amount of time whether we average 8 readings or 8,000 readings."*

3. The "Flight Simulator" Analogy

- **What it explains:** What a Testbench is, and how you validated your custom hardware design.
- **How to pitch it:** *"You can't just 'compile' a custom hardware chip and immediately test it on your wrist; making hardware is unforgiving. So we wrote a **Testbench**, which acts like a flight simulator for our custom chip. Before we ever put our Verilog code onto the real FPGA, we fed the simulator pre-recorded, fake heartbeat data. It allowed us to watch the virtual 1s and 0s firing across the virtual wires to guarantee our math was perfect before touching physical hardware."*

The 4 Hardware Files (Explained for Students)

If a judge asks, "What exactly did you write in Verilog?", here is how you describe the 4 key files:

- **movingaverage8tap.v (The Filter):** This is the O(1) "Smart Scale" mentioned above. It's a custom digital filter that takes in the raw, messy heartbeat wave and smooths it out by averaging the last 8 data points continuously.
- **ppgpeakdetector.v (The Heartbeat Counter):** Once the wave is smoothed out, this module acts like a radar. It uses a 4-step flowchart (a Finite State Machine) to track when the wave goes up, hits the peak, and comes down. By counting the exact time between peaks, it calculates the Heart Rate and Heart Rate Variability completely in hardware.

- `axippgaccelerator.v` (**The Bridge**): The CPU and the custom hardware don't speak the same language. This file is the "translator" wrapper (using the AXI protocol). It allows the ARM CPU to seamlessly read the final heart rate numbers out of our custom hardware block as if it were just reading standard memory.
 - `tbppgsystem.v` (**The Flight Simulator**): This is the Testbench file that isn't actually loaded onto the final device. It's the simulation file we used to verify the other three files worked perfectly in software before we deployed them to the FPGA.
-

10. CLINICAL REFERENCES & BASELINE THRESHOLDS

If a judge asks where the numbers in our Rule-Based Engine (`disasterriskengine.c`) came from, it is critical to explain that the **thresholds** are based on standard clinical guidelines, while the **weighted scoring system** (CTSI/PRSI) is our custom hackathon invention.

1. The Heat Index (Temperature + Humidity)

- **Source:** Robert G. Steadman's 1979 paper "*The Assessment of Sultriness*."
- **Implementation:** We explicitly implemented a "Simplified Steadman Heat Index approximation" in C. It calculates how hot it *feels* based on humidity preventing sweat evaporation, triggering critical alerts when the index exceeds 54°C.

2. Air Pollution (PM2.5) Thresholds

- **Source:** World Health Organization (WHO) Air Quality Guidelines & EPA AQI Standards.
- **Implementation:** Our PM2.5 threshold for "Critical Risk" is strictly set to $>300 \mu\text{g}/\text{m}^3$, which perfectly aligns with the EPA definition of "Hazardous" air quality.

3. Biometric Thresholds (SpO2 & Heart Rate)

- **Source:** Standard clinical triage guidelines (e.g., Mayo Clinic, American Heart Association).
- **Implementation:**
- **Hypoxia:** Our engine flags a High Risk when SpO2 drops below 92% (standard clinical threshold for hypoxemia), and escalates to a maximum Critical Risk below 88%.
- **Tachycardia:** Our engine flags severe cardiovascular strain when the resting heart rate exceeds 120 BPM (during pollution events) or 130 BPM (during extreme heat events).

4. The Scoring Algorithms (CTSI and PRSI)

- **Source:** Custom Hackathon Invention.
- **Implementation:** We took the peer-reviewed clinical thresholds above and created our own weighted point system (0 to 100) — the Cardio-Thermal Strain Index (CTSI) and the Pollution Respiratory Strain Index (PRSI) — specifically for this hardware challenge. This allowed us to generate a continuous mathematical gradient of "Risk" that our Neural Network could easily learn via knowledge distillation.